#AI Assisted Coding

#Assignment –18.3

#K.Sai Charan Teja|| 2303A51819|| Batch:- 8

```
# Generate a Python script to fetch public transport fare details
between two stations using a REST API.

# Tasks:
# - Take source and destination as user inputs and validate them
# - Make an API request using requests library
# - Handle errors such as invalid station names and API downtime (try-
except) # - Parse and display fare details in a clean,
structured format

# Keep the code simple, readable, and beginner-friendly. Include
sample API URL placeholder. import requests # Used to make HTTP
requests to the API

# --- Configuration --# Replace this with the actual API base
URL for fare details.
# This is a placeholder for demonstration purposes.

API_BASE_URL = "https://api.example.com/fare_details"

source_station = input("Enter the source station: ").strip()
destination_station = input("Enter the destination station: ").strip()

if not source_station or not destination_station:
print("Error: Both source and destination cannot be empty.")
else:     print(f"\nFetching fare details from {source_station}
to
{destination_station}...")

    params = {
        'source': source_station,
        'destination': destination_station
    }

    try:
        # Try API call (will fail because it's dummy)
response = requests.get(API_BASE_URL, params=params,
timeout=5)
        response.raise_for_status()
fare_data = response.json()

    except:
        # ✅ Fallback (simulate API response)
fare_data = {
```

```
            "fare": 30,
            "message": "Sample fare (mock data)"
}

    # Display output
    print("\n--- Fare Details ---")
print(f"Source: {source_station}")
print(f"Destination: {destination_station}")
print(f"Fare: ₹{fare_data['fare']}")
print(f"Message: {fare_data['message']}")
print("--------------------")
```

```
Enter the source station: Chennai Central
Enter the destination station: Tambaram

Fetching fare details from Chennai Central to Tambaram...

--- Fare Details ---
Source: Chennai Central
Destination: Tambaram
Fare: ₹30
Message: Sample fare (mock data)
--------------------
```

```
# Generate a Python script using requests, pandas, and sklearn (if
needed) to convert INR to USD, EUR, and GBP using a currency exchange
API.

# Tasks:
# - Take INR amount as user input and validate it
# - Fetch real-time exchange rates using an API (use a sample API URL
placeholder)
# - Handle errors (invalid input, API failure, wrong currency codes)
using try-except
# - Convert INR to USD, EUR, GBP
# - Display results in a clean tabular format (use pandas dataframe or
print table)

# Keep the code simple, clean, and beginner-friendly.
import requests import pandas as pd

API_URL = "https://api.exchangerate-api.com/v4/latest/INR"
TARGET_CURRENCIES = ['USD', 'EUR', 'GBP']

while True:     try:         amt =
float(input("Enter amount in INR: "))         if
amt <= 0:             print("Enter a positive
value.")
```

```
        else:
            break     except:
print("Invalid input. Enter a number.")

print(f"\nFetching exchange rates for {amt:.2f} INR...\n")

try:     res = requests.get(API_URL,
timeout=10)     res.raise_for_status()
data = res.json()

    if 'rates' not in data:
print("Invalid API response.")
else:         rates = data['rates']
result = []

        for cur in TARGET_CURRENCIES:
if cur in rates:
                rate = rates[cur]
converted = amt * rate
                result.append([cur, round(rate, 4), round(converted,
2)])             else:
print(f"{cur} rate not found")

        df = pd.DataFrame(result, columns=["Currency", "Rate (1 INR)",
"Converted Amount"])

        print("Conversion Results:\n")
print(df.to_string(index=False))

except requests.exceptions.ConnectionError:
    print("Connection error. Check internet or API.")
except requests.exceptions.Timeout:
    print("Request timed out.") except
requests.exceptions.HTTPError as e:
print("HTTP error:", e) except Exception
as e:     print("Error:", e)
```

```
Enter amount in INR: 1000 Fetching

exchange rates for 1000.00 INR...

Conversion Results:
Currency  Rate (1 INR)  Converted Amount
     USD        0.0107             10.70
```

```
     EUR        0.0093              9.26
     GBP        0.0080              8.05
# Generate a Python script using requests to fetch GitHub repository
details using the GitHub API.

# Tasks:
# - Take repository owner and repository name as user input
# - Send a GET request to GitHub API
(https://api.github.com/repos/{owner}/{repo})
# - Handle errors such as 404 (repository not found), 403 (rate
limit), and connection issues using try-except
# - Display repository details: name, description, stars, forks, and
open issues in a clean format

# Keep the code simple, clean, and beginner-friendly.
import requests

def get_repo_info(owner, repo):     url =
f"https://api.github.com/repos/{owner}/{repo}"

    try:         response = requests.get(url,
timeout=10)

        if response.status_code == 200:
data = response.json()

            print("\n---       Repository       Details       ---")
print(f"Name                                :    {data.get('name')}")
print(f"Description          :          {data.get('description')}")
print(f"Stars                   :   {data.get('stargazers_count')}")
print(f"Forks                        :    {data.get('forks_count')}")
print(f"Open     Issues     :     {data.get('open_issues_count')}")
print("----------------------------")

        elif response.status_code == 404:
print("Error: Repository not found. Check owner/repo name.")

        elif response.status_code == 403:
print("Error: Rate limit exceeded. Try again later.")

        else:             print(f"HTTP Error:
{response.status_code}")

    except requests.exceptions.ConnectionError:
print("Connection error. Check your internet.")     except
requests.exceptions.Timeout:
        print("Request timed out.")
except Exception as e:
```

```
        print("Unexpected error:", e)


# --- User Input ---
owner = input("Enter repository owner (username): ").strip()
repo = input("Enter repository name: ").strip()

if owner and repo:     get_repo_info(owner, repo) else:
print("Invalid input. Owner and repository name cannot be empty.")
Enter repository owner (username): NithinReddychallagonda
Enter repository name: AI_AC-LAB

--- Repository Details ---
Name        : AI_AC-LAB
Description : None
Stars       : 0
Forks       : 0
Open Issues : 0
---------------------------

import requests
import time

# --- Configuration --# Replace 'YOUR_API_KEY_HERE' with
your actual News API key.
# You can get a free API key from newsapi.org.
API_KEY = "YOUR_API_KEY_HERE"
BASE_URL = "https://newsapi.org/v2/top-headlines"
VALID_CATEGORIES = ['sports', 'technology', 'health']
MAX_RETRIES = 3
RETRY_DELAY = 5 # seconds

# --- Get User Input and Validate --while True:     category =
input("Enter category (sports/technology/health):
").strip().lower()     if category in VALID_CATEGORIES:
break     else:         print(f"Error: Invalid category. Please
choose from {', '.join(VALID_CATEGORIES)}.")

# --- Fetch Headlines Function --def
fetch_headlines(category, api_key):
params = {
        'category': category,
        'language': 'en',
```

```
        'pageSize': 5, # Fetch top 5 headlines
        'apiKey': api_key
    }

    for attempt in range(MAX_RETRIES):
print(f"\nFetching top 5 '{category}' headlines (Attempt
{attempt + 1}/{MAX_RETRIES})...")         try:
response = requests.get(BASE_URL, params=params, timeout=10)
            response.raise_for_status() # Raise HTTPError for bad
responses (4xx or 5xx)
            data = response.json()

            if data.get('status') == 'ok':
                return data.get('articles', [])             elif
data.get('status') == 'error':                 print(f"API Error:
{data.get('message', 'Unknown API error')}")
                if 'apiKeyMissing' in data.get('message', '') or
'apiKeyInvalid' in data.get('message', '') or 'apiKeyDisabled' in
data.get('message', ''):                     print("Please check
your API key. It might be missing, invalid, or disabled.")
                    return None # Critical error, no point retrying

        except requests.exceptions.ConnectionError:
print("Connection error: Please check your internet connection.")
except requests.exceptions.Timeout:             print("Request
timed out: The server took too long to respond.")         except
requests.exceptions.HTTPError as http_err:
print(f"HTTP error occurred: {http_err} - Status Code:
{response.status_code}")
            if response.status_code == 401: # Unauthorized - likely API
key issue
                print("Authentication error: Your API key might be
incorrect or expired.")                 return None
            elif response.status_code == 429: # Too Many Requests
print("Rate limit exceeded: Please wait before making more
requests.")         except requests.exceptions.RequestException as e:
print(f"An unexpected request error occurred: {e}")         except
ValueError as e:             print(f"Error parsing JSON response:
{e}")

        if attempt < MAX_RETRIES - 1:
print(f"Retrying in {RETRY_DELAY} seconds...")
```

```
            time.sleep(RETRY_DELAY)         else:
print("Maximum retries reached. Could not fetch
headlines.")     return None

# --- Main Execution --if API_KEY ==
"YOUR_API_KEY_HERE":     print("Error: API key is
missing. Please replace 'YOUR_API_KEY_HERE' with
your actual News API key.") else:     articles =
fetch_headlines(category, API_KEY)

    if articles:         print(f"\n--- Top 5
{category.capitalize()} Headlines ---")         for i, article in
enumerate(articles):             title = article.get('title', 'No
Title Available')             source = article.get('source',
{}).get('name', 'Unknown Source')
            print(f"{i + 1}. {title} (Source: {source})")
print("-------------------------------------")
    elif articles is not None: # Means articles was empty but no
critical error         print("No headlines found for this
category.")


Enter category (sports/technology/health): sport
Error: Invalid category. Please choose from sports, technology,
health.
Enter category (sports/technology/health): sports
Error: API key is missing. Please replace 'YOUR_API_KEY_HERE' with
your actual News API key.
```